

MANUAL DEL PROGRAMADOR

Plataforma de Compra y Gestión de Créditos Académicos en Línea

UPTC Sogamoso — Seccional Sogamoso

Autores	Juan Steban León Cárdenas (202127502) Julian Leonardo Arias Pita (202128044)
Asignatura	Ingeniería del Software II
Semestre	Primer Semestre 2026
Versión	1.0 — Junio 2026
Repositorio	github.com/JuliaNPit/PlataformadeComprayGestiondeCreditos
Frontend	creditos-uptyc.vercel.app
Backend	creditos-uptyc-backend.onrender.com

1. Introducción

Este Manual del Programador describe la organización técnica del código fuente de la **Plataforma de Créditos UPTC**, los procedimientos para instalar y ejecutar el entorno de desarrollo local, la arquitectura de módulos del backend y frontend, la descripción completa de todos los endpoints de la API REST, y las instrucciones para extender el sistema con nuevos módulos.

El documento está dirigido a desarrolladores que deban mantener, corregir o ampliar el sistema, y complementa la **sección 9.7 del documento principal del proyecto** exigida por las pautas de Ingeniería del Software II.

1.1 Propósito del sistema

La plataforma permite a los estudiantes de la UPTC Seccional Sogamoso comprar créditos académicos para el restaurante universitario, transferirlos entre compañeros, consultar su saldo e historial de transacciones, y radicar solicitudes de soporte (PQRS). El personal administrativo cuenta con un panel de gestión centralizado.

1.2 Alcance técnico

Componente	Tecnología	Despliegue
Frontend (PWA)	React 18 + Vite + Tailwind CSS	Vercel
Backend (API REST)	Node.js 20 + Express 4	Render
Base de datos	PostgreSQL 17 + Prisma ORM 6.19	Render (BD)
Autenticación	JWT + bcryptjs	—
Notificaciones	AWS SES (email) + AWS SNS (SMS)	AWS
Control de versiones	Git + GitHub (GitFlow)	GitHub

2. Requisitos Previos del Entorno

Antes de instalar y ejecutar el proyecto, verificar que el equipo cuente con las siguientes herramientas:

Herramienta	Versión mínima	Descarga / Verificación
Node.js	v20.x LTS	node.js.org — verificar: <code>node --version</code>
npm	v10.x	Incluido con Node.js — verificar: <code>npm --version</code>
PostgreSQL	17.x	postgresql.org — verificar: <code>psql --version</code>
Git	2.x	git-scm.com — verificar: <code>git --version</code>
pgAdmin 4 (opcional)	8.x	pgadmin.org — cliente gráfico para la BD
VS Code (recomendado)	Cualquiera	code.visualstudio.com

Extensiones recomendadas para VS Code: ESLint, Prettier, Prisma, Tailwind CSS IntelliSense, REST Client.

3. Instalación y Configuración Local

3.1 Clonar el repositorio

```
# Clonar el repositorio
git clone https://github.com/JulianNPit/PlataformadeComprayGestiondeCreditos.git
# Entrar al directorio
cd PlataformadeComprayGestiondeCreditos
```

3.2 Configurar la base de datos

Abrir pgAdmin 4 (o usar psql en terminal) y ejecutar:

```
-- Crear la base de datos
CREATE DATABASE creditos_uptc;
```

Verificar que el usuario de PostgreSQL tenga permisos sobre esa base de datos.

3.3 Configurar el backend

```
# Entrar al backend
cd backend
# Instalar dependencias
npm install
# Copiar el archivo de variables de entorno
cp .env.example .env
```

Editar el archivo **backend/.env** con los valores reales:

```
DATABASE_URL="postgresql://postgres:TU_PASSWORD@localhost:5432/creditos_uptc"
JWT_SECRET="uptc_creditos_secret_2026"
JWT_EXPIRES_IN="30m"
PORT=3000
NODE_ENV=development
SIRA_ENABLED=true
FRONTEND_URL="http://localhost:5173"
# Variables AWS (opcionales en desarrollo local)
AWS_REGION="us-east-1"
AWS_ACCESS_KEY_ID="tu_access_key"
AWS_SECRET_ACCESS_KEY="tu_secret_key"
AWS_SES_FROM_EMAIL="no-reply@tudominio.com"
```

Ejecutar las migraciones de Prisma para crear todas las tablas:

```
# Ejecutar migraciones (crea todas las tablas en PostgreSQL)
npx prisma migrate dev
# Opcional: abrir Prisma Studio (interfaz visual de la BD)
npx prisma studio
```

3.4 Iniciar el backend

```
# Modo desarrollo (con nodemon – reinicio automático)
npm run dev

# Modo producción
node src/index.js

# Respuesta esperada:
Servidor corriendo en http://localhost:3000
```

3.5 Configurar el frontend

```
# En otra terminal – entrar al frontend
cd ../frontend

# Instalar dependencias
npm install

# Copiar variables de entorno
cp .env.example .env
```

Editar **frontend/.env**:

```
VITE_API_URL=http://localhost:3000

# Iniciar el servidor de desarrollo
npm run dev

# La aplicación queda disponible en:
http://localhost:5173
```

3.6 Crear usuario administrador

El sistema no tiene un seed de datos inicial. Para crear el primer administrador, registrar un usuario normalmente (código **202127502**, contraseña **123456**) y luego ejecutar en pgAdmin 4 o psql:

```
-- Promover usuario a ADMIN
UPDATE "User" SET role = 'ADMIN' WHERE code = '202127502';
```

4. Estructura del Proyecto

El proyecto está organizado como un monorepo con dos directorios principales: **backend/** (API REST) y **frontend/** (aplicación React).

4.1 Árbol de directorios

PlataformadeComprayGestioneCreditos/		
■■■ backend/		
■ ■■■ prisma/		
■ ■■■ schema.prisma	# Esquema de la base de datos	
■ ■■■ migrations/	# Historial de migraciones SQL	
■ ■■■ src/		
■ ■■■ index.js	# Punto de entrada del servidor	
■ ■■■ adapters/		
■ ■■■ sira.adapter.js	# Simulación del SIRA institucional	
■ ■■■ paymentGateway.adapter.js	# Simulación de pasarela	
■ ■■■ middlewares/		
■ ■■■ auth.middleware.js	# Verificación JWT	
■ ■■■ modules/		
■ ■■■ auth/	# RF01 – Registro y login	
■ ■■■ wallet/	# RF04 – Saldo e historial	
■ ■■■ payments/	# RF02 – Compra de créditos	
■ ■■■ transfers/	# RF03 – Transferencias	
■ ■■■ pqrs/	# RF05 – Sistema de soporte	
■ ■■■ notifications/	# RF07 – Notificaciones	
■ ■■■ admin/	# RF06 – Panel administrativo	
■■■ frontend/		
■■■ src/		
■ ■■■ App.jsx	# Raíz de la aplicación	
■ ■■■ context/AuthContext.jsx	# Estado global de autenticación	
■ ■■■ routes/AppRouter.jsx	# Definición de rutas	
■ ■■■ services/api.js	# Cliente HTTP (axios)	
■ ■■■ pages/		
■ ■■■ Login.jsx	# Pantalla de inicio de sesión	
■ ■■■ Dashboard.jsx	# Panel principal del estudiante	
■ ■■■ Pagos.jsx	# Compra de créditos	
■ ■■■ Transferencias.jsx	# Envío de créditos	
■ ■■■ Historial.jsx	# Historial de transacciones	
■ ■■■ PQRS.jsx	# Sistema de soporte	
■ ■■■ Admin.jsx	# Panel de administración	
■ ■■■ StudentLayout.jsx	# Layout compartido (sidebar + header)	

4.2 Patrón de cada módulo backend

Cada módulo del backend sigue estrictamente el patrón **controlador / servicio / rutas**:

modulo/

```
■■■ modulo.controller.js # Recibe req/res, valida, llama al servicio
■■■ modulo.service.js    # Lógica de negocio + acceso a Prisma
■■■ modulo.routes.js     # Define las rutas y aplica middlewares
```

Los controladores **nunca** acceden directamente a Prisma. Los servicios **nunca** conocen los objetos req ni res de Express. Esta separación facilita las pruebas unitarias y el mantenimiento.

Todas las variables se definen en el archivo **backend/.env**. El archivo **backend/.env.example** en el repositorio contiene la plantilla con valores de ejemplo. **Nunca** subir el archivo .env real al repositorio.

Nota sobre AWS en desarrollo local: Si las variables de AWS no están configuradas, los envíos de email y SMS fallan silenciosamente (fire-and-forget) sin interrumpir las operaciones principales del sistema.

6. Comandos Disponibles

6.1 Backend

Comando	Descripción
npm run dev	Inicia el servidor con nodemon (reinicio automático al guardar)
node src/index.js	Inicia el servidor en modo producción (sin nodemon)
npx prisma migrate dev	Aplica migraciones pendientes y regenera el cliente Prisma
npx prisma migrate deploy	Aplica migraciones en producción (sin modo interactivo)
npx prisma studio	Abre interfaz visual de la base de datos en http://localhost:5555
npx prisma generate	Regenera el cliente Prisma sin ejecutar migraciones
npx prisma db push	Sincroniza el schema directamente (útil para prototipos, sin historial)
node test_aws.js	Ejecuta prueba de envío real de email y SMS via AWS

6.2 Frontend

Comando	Descripción
npm run dev	Inicia el servidor de desarrollo Vite en http://localhost:5173
npm run build	Genera el bundle de producción en la carpeta dist/
npm run preview	Sirve localmente el bundle de producción para pruebas

La URL base en desarrollo local es **http://localhost:3000/api**. En producción: **https://creditos-uptc-backend.onrender.com/api**.

7.1 Autenticación — /api/auth

Método	Ruta	Auth	Body / Parámetros	Respuesta exitosa
POST	/register	—	code, email, password	201 { token, user }
POST	/login	—	code, password	200 { token, user }

```
{
  "code": "202127502",
  "email": "juan@uptc.edu.co",
  "password": "123456"
}
```

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR...",
  "user": {
    "id": "clxyz...",
    "name": "Juan Steban León Cárdenas",
    "code": "202127502",
    "role": "STUDENT"
  }
}
```

Método	Ruta	Auth	Parámetros Query	Respuesta
GET	/saldo	■	—	200 { balance, updatedAt }
GET	/historial	■	tipo?, fechaInicio?, fechaFin?, page?, limit?	200 { transactions[], total, page, totalPages }

PURCHASE	TRANSFER_SENT	TRANSFER_RECEIVED	CONSUMPTION
----------	---------------	-------------------	-------------

Página 10

Método	Ruta	Auth	Body / Parámetros	Respuesta
GET	/opciones	■	—	200 { opciones: [{ credits, precio }] }
POST	/iniciar	■	cantidadCreditos, metodo	200 { datos: { creditsComprados, precioPagado, metodo } }

Valores válidos para metodo: NEQUI | PSE | TARJETA

Valores válidos para cantidadCreditos: 1 | 2 | 3 | 5 | 10 | 20

Tabla de precios vigente:

Créditos	Precio (COP)
1	\$3.900
2	\$7.800
3	\$11.700
5	\$19.500
10	\$39.000
20	\$78.000

7.4 Transferencias — /api/transfers

Método	Ruta	Auth	Body	Respuesta
POST	/	■	destinatarioCode, cantidad	200 { mensaje, transaccionId, cantidad, destinatario }

// Body de ejemplo:

```
{
  "destinatarioCode": "202128044",
  "cantidad": 2
}
```

7.5 PQRS — /api/pqrs

Método	Ruta	Auth	Body / Params	Respuesta
POST	/	■	type, title, description	201 { ticket }
GET	/	■	—	200 { tickets[] }
GET	/:id	■	id en URL	200 { ticket }
GET	/admin/todos	■■	—	200 { tickets[] } (todos los usuarios)
PATCH	/:id/responder	■■	message, status	200 { ticket actualizado }

Valores válidos para type: PETICION | QUEJA | RECLAMO | SUGERENCIA

Valores válidos para status (responder): EN_REVISION | RESUELTO | CERRADO

7.6 Notificaciones — /api/notifications

Método	Ruta	Auth	Descripción
GET	/	■	Obtiene las últimas 50 notificaciones del usuario autenticado
PUT	/:id/leer	■	Marca una notificación específica como leída
PUT	/leer/todas	■	Marca todas las notificaciones del usuario como leídas

7.7 Administración — /api/admin

Método	Ruta	Auth	Descripción
GET	/metricas	■■	Retorna: totalUsuarios, totalTransacciones, transaccionesHoy, creditosEnCirculacion
GET	/usuarios	■■	Lista todos los usuarios con su saldo de wallet
GET	/transacciones	■■	Lista los últimos 100 pagos del sistema
PATCH	/usuarios/:id/toggle	■■	Activa o desactiva un usuario (toggle de isActive)

8. Modelos de Base de Datos (Prisma)

La base de datos PostgreSQL es gestionada completamente a través de **Prisma ORM**. El schema fuente está en `backend/prisma/schema.prisma`. A continuación se describe cada modelo con sus campos principales.

Modelo	Campos principales	Relaciones
User	id, code (único), email, phone?, name, password, refreshToken, createdAt, updatedAt	wallet (1:1), payments, sentTransfers, receivedTransfers, pqrsCreated
Wallet	id, userId, balance (créditos), totalSpent	user (1:1), transactions[]
Transaction	id, walletId, type, amount, description, status	wallet, transfer?, payment?
Payment	id, userId, amount (COP), credits, method, status, gatewayTransactionId?	user, transactions[]
Transfer	id, senderId, receiverId, amount, status	sender (User), receiver (User), transactions[]
PQRS	id, ticketNumber (único), userId, type, title, description, status, priority, createdAt, updatedAt	user, status, priority, updates[]
PQRSUpdate	id, pqrsId, message, status	pqrs
Notification	id, userId, type, title, message, read	—
AuditLog	id, userId?, action, entity, entityId, oldValue?, newValue?	—
SystemConfig	id, creditPrice, minTransferAmount, maxTransferAmount, ...	—

8.1 Enumeraciones del esquema

Enum	Valores
UserRole	STUDENT · ADMIN · OPERATOR
TransactionType	PURCHASE · TRANSFER_SENT · TRANSFER_RECEIVED · CONSUMPTION
TransactionStatus	PENDING · COMPLETED · FAILED · REVERSED
PaymentMethod	PSE · NEQUI · TARJETA
PaymentStatus	PENDING · APPROVED · REJECTED · CANCELLED
TransferStatus	PENDING · COMPLETED · FAILED · CANCELLED
PQRSType	PETICION · QUEJA · RECLAMO · SUGERENCIA
PQRSStatus	ABIERTO · EN_REVISION · RESUELTO · CERRADO
Priority	BAJA · NORMAL · ALTA · URGENTE
NotificationType	TRANSFER_RECEIVED · PURCHASE_COMPLETED · LOW_BALANCE · PQRS_UPDATE · SYSTEM_MESSAGE

9. Cómo Agregar un Nuevo Módulo

Todos los módulos del sistema siguen el mismo patrón de tres archivos. Para agregar un módulo nuevo (por ejemplo, **consumos** para registrar consumos en el restaurante), seguir estos pasos:

Paso 1 — Crear la carpeta y los tres archivos

```
mkdir backend/src/modules/consumos
touch backend/src/modules/consumos/consumos.routes.js
touch backend/src/modules/consumos/consumos.controller.js
touch backend/src/modules/consumos/consumos.service.js
```

Paso 2 — Definir el servicio (lógica de negocio)

```
// consumos.service.js
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

const registrarConsumo = async (userId, { cantidad }) => {
  // Lógica aquí – acceso a Prisma directo
  const wallet = await prisma.wallet.findUnique({ where: { userId } });
  if (!wallet || wallet.balance < cantidad)
    throw new Error('Saldo insuficiente');
  // ... actualizar balance y crear Transaction ...
};

module.exports = { registrarConsumo };
```

Paso 3 — Definir el controlador

```
// consumos.controller.js
const consumosService = require('./consumos.service');

const registrar = async (req, res) => {
  try {
    const { cantidad } = req.body;
    const result = await consumosService.registrarConsumo(req.user.userId, { cantidad });
    res.json({ mensaje: 'Consumo registrado', datos: result });
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
};

module.exports = { registrar };
```

Paso 4 — Definir las rutas

```
// consumos.routes.js
const express = require('express');
const router = express.Router();
```

```
const authMiddleware = require('../../middlewares/auth.middleware');
const consumosController = require('../consumos.controller');

router.use(authMiddleware); // Todas las rutas requieren JWT
router.post('/', consumosController.registrar);

module.exports = router;
```

Paso 5 — Registrar las rutas en index.js

```
// backend/src/index.js – agregar estas dos líneas
const consumosRoutes = require('../modules/consumos/consumos.routes');

// Después de los otros app.use():
app.use('/api/consumos', consumosRoutes);
```

Paso 6 — Agregar modelos en Prisma si es necesario

Si el nuevo módulo necesita una tabla nueva, editar prisma/schema.prisma y ejecutar:

```
npx prisma migrate dev --name add_consumos
```

10. Instrucciones de Despliegue

10.1 Backend en Render

Paso	Acción
1	Crear cuenta en render.com y conectar el repositorio de GitHub
2	Crear un nuevo servicio Web. Root Directory: backend/
3	Build Command: npm install && npx prisma generate
4	Start Command: node src/index.js
5	En la pestaña Environment, agregar todas las variables del .env (DATABASE_URL, JWT_SECRET, etc.)
6	Crear una base de datos PostgreSQL gratuita en Render y copiar la Internal Database URL como DATABASE_URL
7	Ejecutar las migraciones: en el Shell de Render correr npx prisma migrate deploy
8	La URL pública quedará en formato: https://nombre-servicio.onrender.com

10.2 Frontend en Vercel

Paso	Acción
1	Crear cuenta en vercel.com y conectar el repositorio de GitHub
2	Seleccionar el repositorio. Root Directory: frontend/
3	Framework Preset: Vite (detectado automáticamente)
4	En Environment Variables, agregar: VITE_API_URL = https://nombre-backend.onrender.com
5	El archivo vercel.json ya incluye la regla de rewrite para SPA: { "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
6	Hacer deploy. La URL quedará en formato: https://nombre-proyecto.vercel.app

10.3 Actualizar la URL del API en el frontend

El cliente HTTP del frontend lee la URL base desde la variable de entorno. Editar frontend/src/services/api.js si se cambia de proveedor:

```
// frontend/src/services/api.js
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL + '/api',
  // o hardcodeado en producción:
  // baseURL: 'https://creditos-uptc-backend.onrender.com/api',
});
```

11. Usuarios de Prueba (SIRA Simulado)

El adaptador SIRA (backend/src/adapters/sira.adapter.js) valida los códigos estudiantiles contra una lista estática que simula el sistema institucional real. Los siguientes usuarios están disponibles para pruebas:

Código	Nombre	Programa	Estado	Contraseña
202127502	Juan Steban León Cárdenas	Ing. de Sistemas	Activo	123456
202128044	Julian Leonardo Arias Pita	Ing. de Sistemas	Activo	123456
202011111	María García López	Ing. Industrial	Activo	123456
202022222	Carlos Pérez Mora	Ing. de Sistemas	Activo	123456
202033333	Ana Rodríguez Silva	Adm. de Empresas	Activo	123456
202044444	Luis Martínez Torres	Ing. de Sistemas	INACTIVO	N/A

Nota: Para agregar nuevos usuarios de prueba al SIRA simulado, editar el array `estudiantesActivos` en `backend/src/adapters/sira.adapter.js`. En producción, este adaptador sería reemplazado por una llamada HTTP real al sistema institucional, sin modificar la lógica de negocio.

12. Control de Versiones y Flujo de Git

El proyecto usa **GitFlow simplificado** para un equipo de dos personas. El repositorio está en GitHub en la URL indicada en la portada.

12.1 Ramas principales

Rama	Propósito
main	Código en producción. Solo recibe merges de develop cuando hay una versión lista.
develop	Integración continua. Todas las features se fusionan aquí antes de ir a main.
feature/*	Una rama por funcionalidad. Se crea desde develop y se fusiona a develop.

12.2 Flujo de trabajo diario

```
# 1. Actualizar develop
git checkout develop && git pull origin develop

# 2. Crear rama de la funcionalidad
git checkout -b feature/nombre-funcionalidad

# 3. Trabajar y hacer commits con prefijos semánticos:
#   feat:   nueva funcionalidad
#   fix:    corrección de bug
#   docs:   documentación
#   chore:  configuración, dependencias
#   refactor: reorganización de código sin cambio funcional

git add .
git commit -m "feat: agregar endpoint de consumos"

# 4. Subir la rama y abrir Pull Request hacia develop
git push origin feature/nombre-funcionalidad
```

13. Convenciones de Código y Manejo de Errores

13.1 Respuestas de error de la API

Todos los errores se devuelven con la clave error en el cuerpo JSON:

```
// Respuesta de error estándar
{ "error": "Mensaje descriptivo del problema" }

// Códigos HTTP usados:
400 – Error de validación o de negocio (ej: saldo insuficiente)
401 – No autenticado (token ausente)
403 – Token inválido, expirado, o rol insuficiente
404 – Recurso no encontrado
500 – Error interno del servidor
```

13.2 Transacciones atómicas con Prisma

Las operaciones que involucran múltiples cambios en la base de datos (compra, transferencia) usan `prisma.$transaction([])` para garantizar atomicidad. Si cualquier operación falla, todas se revierten:

```
// Patrón de transacción en transfers.service.js
const result = await prisma.$transaction(async (tx) => {
  await tx.wallet.update({ where: { userId: senderId },
    data: { balance: { decrement: cantidad } } });
  await tx.wallet.update({ where: { userId: receiverId },
    data: { balance: { increment: cantidad } } });
  const transfer = await tx.transfer.create({ data: { ... } });
  return transfer;
});
```

13.3 Notificaciones fire-and-forget

Los envíos de email (AWS SES) y SMS (AWS SNS) se ejecutan sin `await` y con `.catch(() => {})`, de modo que si el servicio de notificaciones falla, la operación principal (compra, transferencia) ya fue confirmada y el usuario no ve un error:

```
// Patrón fire-and-forget para notificaciones
enviarEmail({ to: user.email, subject, html }).catch(() => {});
enviarSMS({ telefono: user.phone, mensaje }).catch(() => {});
```

14. Glosario Técnico

Término	Definición
API REST	Interfaz de programación de aplicaciones que sigue los principios REST (stateless, recursos identificados por URL).
JWT	JSON Web Token. Estándar para transmitir información entre partes como un objeto JSON firmado digitalmente.
ORM	Object-Relational Mapper. Herramienta que mapea objetos del lenguaje de programación a tablas de la base de datos.
PWA	Progressive Web App. Aplicación web con capacidades nativas: instalable, funciona offline, con manifest y service workers.
bcrypt	Algoritmo de hash adaptativo para contraseñas. Factor de costo: 10 rounds en este proyecto.
Prisma	ORM de Node.js con schema declarativo, cliente con tipado y sistema de migraciones basado en SQL.
SIRA	Sistema de Información de Registro Académico. Sistema institucional de la UPTC que contiene el registro de estudiantes.
GitFlow	Modelo de ramificación de Git que define ramas main, develop y feature/*. Aquí usado en versión simplificada.
CORS	Cross-Origin Resource Sharing. Mecanismo de seguridad del navegador que controla qué orígenes pueden hacer solicitudes a recursos de otro origen.
AWS SES	Amazon Simple Email Service. Servicio de envío masivo de emails de AWS. Usado para notificaciones transaccionales.
AWS SNS	Amazon Simple Notification Service. Servicio de mensajería de AWS. Usado para envío de SMS al teléfono de los usuarios.
Render	Plataforma cloud PaaS usada para desplegar el backend Node.js y la base de datos PostgreSQL de forma gratuita.
Vercel	Plataforma de despliegue para frontends. Detecta automáticamente proyectos Vite/React y los despliega con CDN.
Atomic transaction	Operación que se ejecuta completamente o no se ejecuta. Usada en transferencias para garantizar que ningún dinero se pierda.
Fire-and-forget	Patrón donde se dispara una operación asíncrona sin esperar su resultado, para no bloquear el flujo principal.

Manual del Programador — Plataforma de Créditos UPTC · Versión 1.0 · Junio 2026
Juan Steban León Cárdenas (202127502) · Julian Leonardo Arias Pita (202128044)
Ingeniería del Software II — UPTC Sogamoso